

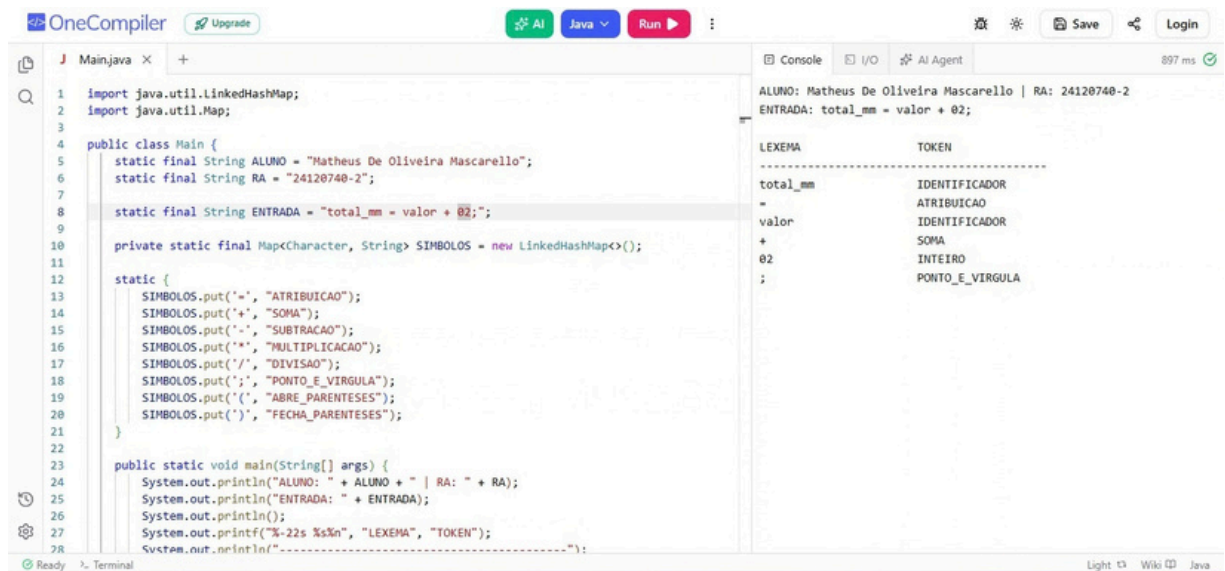
ExercíciosJava

ProfessorMunif.

Aluno: Gabriel Santos Furlanette RA:

24000873-2

Exercício 1:



```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 public class Main {
5     static final String ALUNO = "Matheus De Oliveira Mascarello";
6     static final String RA = "24120740-2";
7
8     static final String ENTRADA = "total_mm = valor + 02;";
9
10    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();
11
12    static {
13        SIMBOLOS.put('=', "ATRIBUICAO");
14        SIMBOLOS.put('+', "SOMA");
15        SIMBOLOS.put('-', "SUBTRACAO");
16        SIMBOLOS.put('*', "MULTIPLICACAO");
17        SIMBOLOS.put('/', "DIVISAO");
18        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
19        SIMBOLOS.put('(', "ABRE_PARENTESES");
20        SIMBOLOS.put(')', "FECHA_PARENTESES");
21    }
22
23    public static void main(String[] args) {
24        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
25        System.out.println("ENTRADA: " + ENTRADA);
26        System.out.println();
27        System.out.printf("%-22s %s\n", "LEXEMA", "TOKEN");
28        System.out.println("-----");
29
30        for (int i = 0; i < ENTRADA.length(); i++) {
31            char c = ENTRADA.charAt(i);
32            String token = SIMBOLOS.get(c);
33            System.out.printf("%-12s %s\n", c, token);
34        }
35    }
36 }
```

ALUNO: Matheus De Oliveira Mascarello | RA: 24120740-2
ENTRADA: total_mm = valor + 02;

LEXEMA	TOKEN
total_mm	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
02	INTEIRO
;	PONTO_E_VIRGULA

Exercício 2:

```
1 import java.util.Arrays;
2 import java.util.HashSet;
3 import java.util.Set;
4
5 public class Main {
6     static final String ALUNO = "Matheus De Oliveira Mascarello";
7     static final String RA = "24120740-2";
8
9     static final String IDENTIFICADOR_PERSONALIZADO = "#";
10
11     static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>{
12         Arrays.asList("int", "if", "else", "while", "return")
13     };
14
15     public static void main(String[] args) {
16         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
17         executarTeste("TESTE A - entrada compacta",
18             "int total=valor+10;");
19         executarTeste("TESTE B - espaços e comentário",
20             "int total = valor + 10; // este comentario sera ignorado");
21         executarTeste("TESTE C - reservadas e identificadores",
22             "int inteiro = 0; while (inteiro < intValue) inteiro = inteiro + 1;");
23         executarTeste("TESTE C2 - identificador personalizado",
24             "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
25         executarTeste("TESTE D - caractere inválido",
26             "int valor# = 1;");
27     }
28 }
```

Console Output:

```
inteiro      LETRA      IDENTIFICADOR
+            SIMBOLO    SOMA
1            DIGITO     INTEIRO
;            SIMBOLO    PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===
ENTRADA: int MM = 10;
LEXEMA      CLASSE     TOKEN
-----
int          LETRA      PALAVRA_RESERVADA
MM          LETRA      IDENTIFICADOR
=           SIMBOLO    ATRIBUICAO
10          DIGITO     INTEIRO
;           SIMBOLO    PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===
ENTRADA: int valor# = 1;
LEXEMA      CLASSE     TOKEN
-----
int          LETRA      PALAVRA_RESERVADA
valor       LETRA      IDENTIFICADOR
#           DESCONHECIDO ERRO_LEXICO
=           SIMBOLO    ATRIBUICAO
1           DIGITO     INTEIRO
;           SIMBOLO    PONTO_E_VIRGULA
```

Exercício 3:

```
1 public class Main {
2     static final String ALUNO = "Matheus De Oliveira Mascarello";
3     static final String RA = "24120740-2";
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
12            System.out.println("Resultado: " + total);
13        }
14    }
15 }
16 }
```

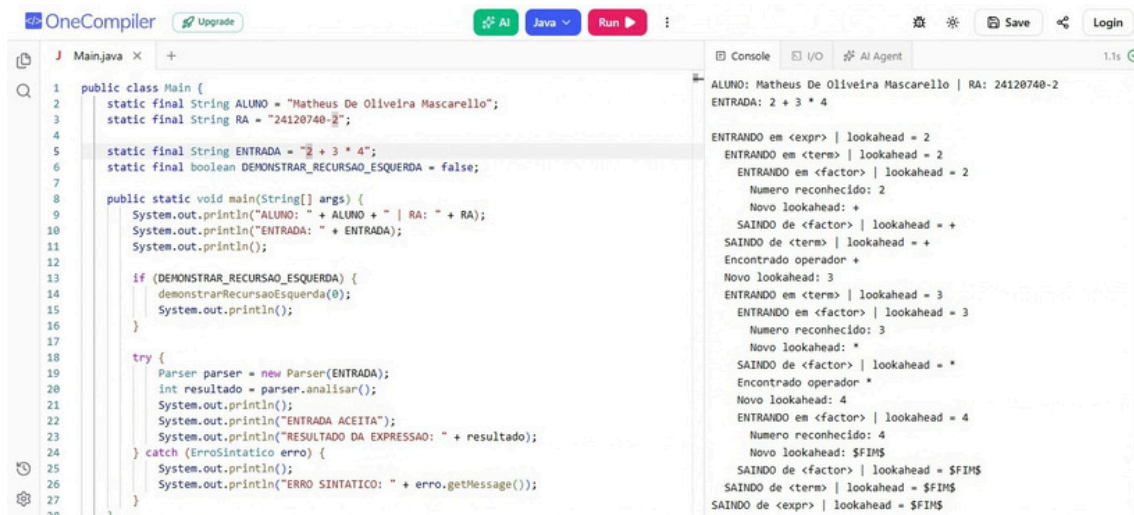
Console Output (Errors):

```
Main.java:8: error: ';' expected
int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e
virgula
^
Main.java:10: error: ')' expected
if (total > 20 { // CORRIGIR 2: feche corretamente o
parenteses
^
Main.java:11: error: unclosed string literal
System.out.println("Programa corrigido!"); // CORRIGIR
3: feche o texto entre aspas
^
3 errors
```

Console Output (Result):

```
ALUNO: Matheus De Oliveira Mascarello | RA: 24120740-2
Programa corrigido!
Resultado: 30
```

Exercício 4:



The screenshot displays the OneCompiler web IDE. The editor on the left contains a Java file named `Main.java` with the following code:

```
1 public class Main {
2     static final String ALUNO = "Matheus De Oliveira Mascarello";
3     static final String RA = "24120740-2";
4
5     static final String ENTRADA = "2 + 3 * 4";
6     static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;
7
8     public static void main(String[] args) {
9         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
10        System.out.println("ENTRADA: " + ENTRADA);
11        System.out.println();
12
13        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
14            demonstrarRecursoEsquerda(0);
15            System.out.println();
16        }
17
18        try {
19            Parser parser = new Parser(ENTRADA);
20            int resultado = parser.analisar();
21            System.out.println();
22            System.out.println("ENTRADA ACEITA");
23            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
24        } catch (ErroSintatico erro) {
25            System.out.println();
26            System.out.println("ERRO SINTATICO: " + erro.getMessage());
27        }
28    }
29 }
```

The console on the right shows the output of the program:

```
ALUNO: Matheus De Oliveira Mascarello | RA: 24120740-2
ENTRADA: 2 + 3 * 4

ENTRANDO em <expr> | lookahead = 2
ENTRANDO em <term> | lookahead = 2
ENTRANDO em <factor> | lookahead = 2
Numero reconhecido: 2
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead: *
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: $FIM$
SAINDO de <factor> | lookahead = $FIM$
SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$
```

Exercício 5: